

# Technical Report: Minimal Mixture-of-Experts Language Model with Muon Optimizer

Vuk Rosić<sup>1,2</sup>

<sup>1</sup>Open Superintelligence Lab

<sup>2</sup>Óbuda University

November 29, 2025

## Abstract

This technical report presents a minimal yet efficient implementation of a Mixture-of-Experts (MoE) based language model. The model employs a decoder-only Transformer architecture enhanced with sparse MoE layers for improved parameter efficiency. Training is conducted using a hybrid optimization strategy combining the Muon optimizer for 2D weight matrices with AdamW for other parameters. The model is trained on the SmolLM Cosmopedia dataset with 384-dimensional embeddings, 6 layers, and 8 experts per MoE layer with top-2 routing. Key features include Rotary Positional Embeddings (RoPE), RMSNorm, and load balancing mechanisms to ensure efficient expert utilization. This implementation demonstrates how modern architectural innovations can be combined to create compact, trainable language models.

## 1 Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities across a wide range of natural language processing tasks. However, the computational costs associated with training and deploying these models remain a significant challenge. Mixture-of-Experts (MoE) architectures offer a promising solution by enabling conditional computation, where only a subset of model parameters is activated for each input token.

This report describes a minimal implementation of an MoE-based language model that incorporates several modern architectural innovations:

- **Sparse Mixture of Experts:** Each feedforward layer is replaced with a collection of expert networks, with only the top-2 experts activated per token.
- **Hybrid Optimization:** The Muon optimizer handles 2D weight matrices (benefiting from Newton-Schulz orthogonalization), while AdamW handles embeddings and normalization parameters.
- **Rotary Positional Embeddings:** RoPE provides efficient positional encoding without requiring learned position embeddings.
- **RMSNorm:** Root Mean Square normalization for improved training stability with lower computational cost than LayerNorm.

The goal of this implementation is to demonstrate how these components can be combined into a compact, efficient, and trainable language model suitable for educational purposes and rapid prototyping.

## 2 Model Architecture

### 2.1 Overview

The model follows a decoder-only Transformer architecture, similar to GPT-style models. The key architectural components are:

1. Token embedding layer
2.  $N$  transformer blocks (each containing attention and MoE feedforward)
3. Output normalization
4. Language modeling head (weight-tied with token embeddings)

### 2.2 Hyperparameters

The model configuration is defined with the following hyperparameters:

| Parameter                              | Value  |
|----------------------------------------|--------|
| Model dimension ( $d_{\text{model}}$ ) | 384    |
| Number of layers ( $N$ )               | 6      |
| Number of attention heads ( $H$ )      | 8      |
| Head dimension ( $d_k$ )               | 48     |
| Feedforward dimension ( $d_{ff}$ )     | 1536   |
| Number of experts ( $E$ )              | 8      |
| Expert top-k                           | 2      |
| Maximum sequence length                | 512    |
| Vocabulary size                        | 49,152 |
| Dropout rate                           | 0.1    |

Table 1: Model architecture hyperparameters

### 2.3 Attention Mechanism

The model uses Multi-Head Attention (MHA) with Rotary Positional Embeddings. The attention mechanism can be formulated as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

where  $Q$ ,  $K$ , and  $V$  are the query, key, and value matrices. The queries and keys are augmented with RoPE before computing attention scores:

$$Q_{\text{rope}} = \text{RoPE}(Q), \quad K_{\text{rope}} = \text{RoPE}(K) \quad (2)$$

RoPE applies rotation matrices to embed absolute positional information while maintaining relative positional relationships. The implementation uses PyTorch’s Flash Attention via `scaled_dot_product_attention` with causal masking for efficient autoregressive generation.

## 2.4 Mixture of Experts Layer

Each transformer block contains a Mixture of Experts layer instead of a traditional feedforward network. The MoE layer consists of:

- **Router network:** A learned linear projection that computes expert selection scores
- **Expert networks:**  $E = 8$  independent feedforward networks
- **Top-k gating:** Selects the top-2 experts per token

### 2.4.1 Expert Network

Each expert is a simple two-layer feedforward network:

$$\text{Expert}(x) = W_2 \cdot \text{Dropout}(\text{SiLU}(W_1 x)) \quad (3)$$

where  $W_1 \in \mathbb{R}^{d_{ff} \times d_{\text{model}}}$  and  $W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{ff}}$ , and SiLU is the Sigmoid Linear Unit activation function.

### 2.4.2 Router Mechanism

For each input token  $x$ , the router computes logits for all experts:

$$r = W_g x + \epsilon \quad (4)$$

where  $W_g \in \mathbb{R}^{E \times d_{\text{model}}}$  is the gating network and  $\epsilon \sim \mathcal{N}(0, \sigma^2)$  is Gaussian noise (applied during training for exploration). The top-2 experts are selected:

$$\text{indices, weights} = \text{TopK}(\text{softmax}(r), k = 2) \quad (5)$$

The final MoE output is the weighted combination of selected experts:

$$\text{MoE}(x) = \sum_{i \in \text{top-k}} w_i \cdot \text{Expert}_i(x) \quad (6)$$

### 2.4.3 Load Balancing Loss

To prevent the model from routing all tokens to a few experts, a load balancing auxiliary loss is added:

$$\mathcal{L}_{\text{aux}} = \lambda \cdot E \sum_{i=1}^E f_i \cdot p_i \quad (7)$$

where  $f_i$  is the fraction of tokens routed to expert  $i$ ,  $p_i$  is the average routing probability to expert  $i$ , and  $\lambda = 0.01$  is the balancing weight. This loss encourages uniform distribution of tokens across experts.

## 2.5 Normalization and Residual Connections

Each transformer block uses pre-normalization with RMSNorm:

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}} \cdot \gamma \quad (8)$$

where  $\gamma$  is a learned scale parameter. The block follows a standard residual connection pattern:

$$h_1 = x + \text{Dropout}(\text{Attention}(\text{RMSNorm}(x))) \quad (9)$$

$$h_2 = h_1 + \text{Dropout}(\text{MoE}(\text{RMSNorm}(h_1))) \quad (10)$$

### 3 Training Methodology

#### 3.1 Dataset

The model is trained on the SmolLM Cosmopedia dataset, a high-quality subset of educational text. The dataset is processed as follows:

- **Total documents:** 10,000
- **Train/validation split:** 90/10
- **Tokenization:** SmolLM-135M tokenizer (49,152 vocabulary)
- **Sequence length:** 512 tokens
- **Processing:** Documents are concatenated and chunked into fixed-length sequences

#### 3.2 Optimization Strategy

A hybrid optimization approach is employed, combining two optimizers:

##### 3.2.1 Muon Optimizer

The Muon optimizer is applied to all 2D weight matrices (excluding embeddings and normalization parameters). Muon leverages Newton-Schulz iterations for orthogonalization, which can provide better conditioning for weight matrices. Configuration:

- Learning rate: 0.07
- Momentum: 0.85

##### 3.2.2 AdamW Optimizer

AdamW handles 1D parameters (embeddings, biases, normalization scales):

- Learning rate: 0.007
- Weight decay: 0.3

#### 3.3 Training Configuration

#### 3.4 Learning Rate Schedule

A cosine annealing schedule with warmup is used:

$$\eta(t) = \begin{cases} \frac{t}{T_{\text{warmup}}} \cdot \eta_{\text{max}} & \text{if } t < T_{\text{warmup}} \\ \eta_{\text{min}} + \frac{1}{2}(\eta_{\text{max}} - \eta_{\text{min}}) \left( 1 + \cos \left( \pi \frac{t - T_{\text{warmup}}}{T_{\text{max}} - T_{\text{warmup}}} \right) \right) & \text{otherwise} \end{cases} \quad (11)$$

where  $\eta_{\text{max}}$  is the base learning rate,  $\eta_{\text{min}} = 0.1\eta_{\text{max}}$ ,  $T_{\text{warmup}} = 50$  steps, and  $T_{\text{max}} = 1000$  steps.

| Parameter                   | Value           |
|-----------------------------|-----------------|
| Batch size                  | 24              |
| Gradient accumulation steps | 4               |
| Effective batch size        | 96              |
| Maximum training steps      | 1,000           |
| Warmup ratio                | 0.05 (50 steps) |
| Gradient clipping           | 1.0             |
| Mixed precision             | FP16 (AMP)      |
| Learning rate schedule      | Cosine decay    |
| Evaluation frequency        | Every 10 steps  |

Table 2: Training hyperparameters

### 3.5 Loss Function

The total loss combines cross-entropy for language modeling with the MoE auxiliary loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{aux}} \quad (12)$$

where the cross-entropy loss is computed autoregressively:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{T} \sum_{t=1}^T \log P(x_t | x_{<t}) \quad (13)$$

## 4 Implementation Details

### 4.1 Framework and Libraries

The implementation uses:

- **PyTorch**: Core deep learning framework
- **torch tune**: For Rotary Positional Embeddings
- **Transformers (HuggingFace)**: For tokenizer
- **Datasets (HuggingFace)**: For data loading

### 4.2 Parameter Count

The model has approximately:

- **Total parameters**: ~30M (varies based on exact vocabulary size)
- **Active parameters per forward pass**: ~20M (65-70% of total)
- **Expert parameters**: ~10M

The sparse activation of experts provides significant computational savings while maintaining model capacity.

### 4.3 Weight Initialization

All weights are initialized using a Gaussian distribution:

- Linear layers:  $\mathcal{N}(0, 0.02^2)$
- Embeddings:  $\mathcal{N}(0, 0.02^2)$
- Biases: Zeros (where applicable)

The output language modeling head shares weights with the token embedding layer (weight tying), reducing parameters and improving sample efficiency.

### 4.4 Computational Efficiency

Several techniques are employed for efficient training:

- **Automatic Mixed Precision (AMP):** FP16 computation with FP32 master weights
- **Flash Attention:** Memory-efficient attention via PyTorch’s SDPA
- **Gradient Accumulation:** Simulates larger batch sizes with limited memory
- **Persistent Data Loaders:** Reuses worker processes across epochs

## 5 Code Structure

The implementation is organized into modular components:

- `configs/moe_config.py`: Configuration dataclass with all hyperparameters
- `models/moe_llm.py`: Main model class (MoEMinimalLLM)
- `models/layers.py`: Transformer block, attention mechanisms
- `models/components.py`: MoE layer, router, experts
- `training/trainer.py`: Training loop, optimization, evaluation
- `optimizers/muon.py`: Muon optimizer implementation
- `train_moe.py`: Main training script

This modular design facilitates experimentation and extension of the architecture.

## 6 Evaluation Metrics

The model is evaluated using standard language modeling metrics:

- **Validation Loss:** Cross-entropy loss on held-out data
- **Validation Accuracy:** Next-token prediction accuracy
- **Perplexity:**  $\text{PPL} = \exp(\mathcal{L}_{\text{CE}})$

Metrics are computed every 10 training steps to monitor convergence.

## 7 Conclusion

This technical report has presented a minimal Mixture-of-Experts language model implementation that combines several modern architectural innovations. The model demonstrates:

- Efficient parameter usage through sparse expert activation
- Effective training via hybrid Muon/AdamW optimization
- Scalable architecture suitable for educational and research purposes
- Clean, modular code organization for easy experimentation

The implementation serves as a foundation for understanding MoE architectures and can be extended in various directions:

- Scaling to larger model sizes and expert counts
- Exploring alternative attention mechanisms (e.g., Multi-Head Latent Attention)
- Implementing more sophisticated routing strategies
- Fine-tuning on downstream tasks

The complete source code is available in the project repository, providing a practical reference for implementing efficient language models with mixture-of-experts architectures.

## References

1. Vaswani, A., et al. (2017). "Attention is All You Need." *NeurIPS*.
2. Shazeer, N., et al. (2017). "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer." *ICLR*.
3. Su, J., et al. (2021). "RoFormer: Enhanced Transformer with Rotary Position Embedding." *arXiv*.
4. Zhang, B., & Sennrich, R. (2019). "Root Mean Square Layer Normalization." *NeurIPS*.
5. Loshchilov, I., & Hutter, F. (2019). "Decoupled Weight Decay Regularization." *ICLR*.
6. Jordan, K., Jin, Y., Boza, V., You, J., Cesista, F., Newhouse, L., & Bernstein, J. (2024). "Muon: An optimizer for hidden layers in neural networks." <https://kellerjordan.github.io/posts/muon/>